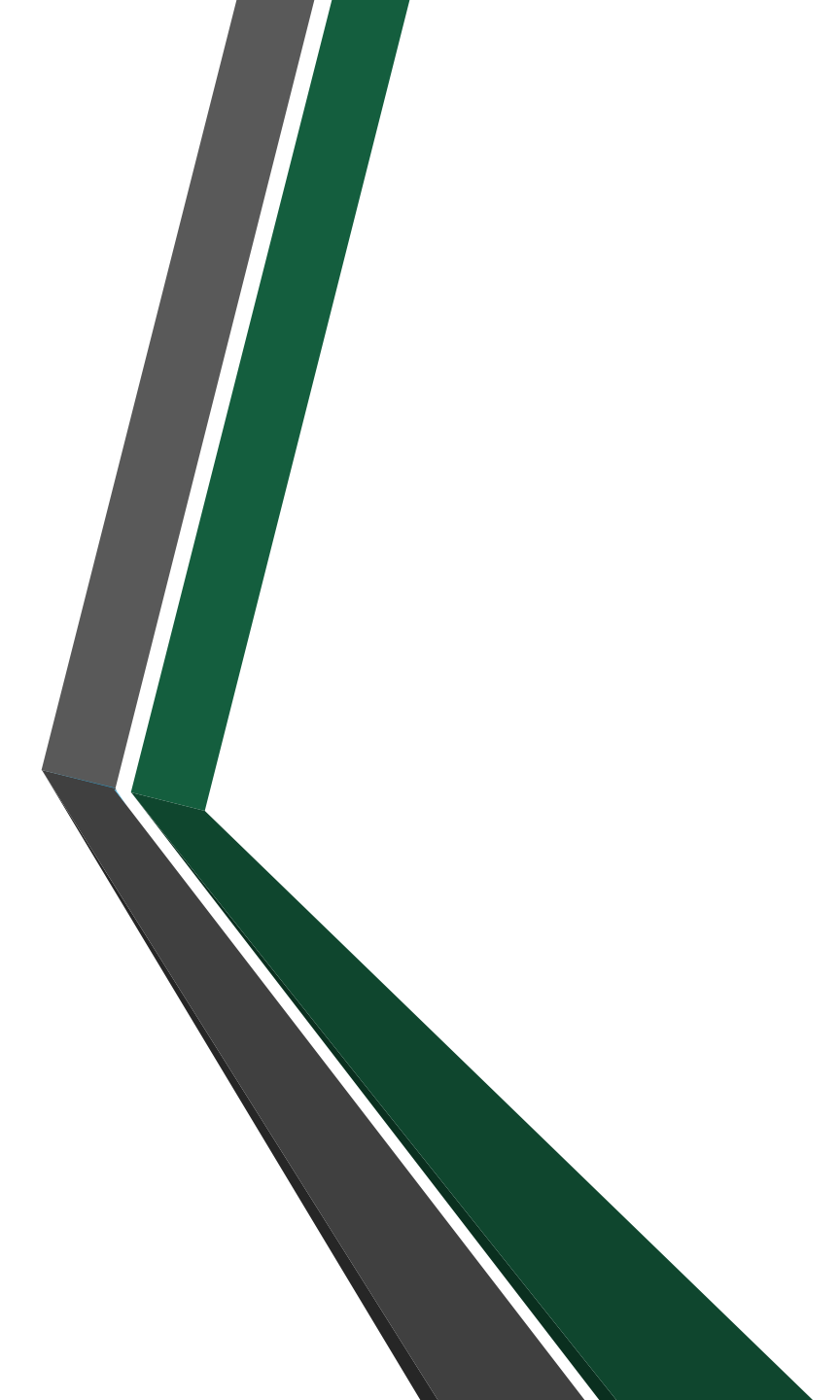




BAZE DE DATE

CURS 10

Necesitatea normalizării

- În momentul proiectării și modelării unei baze de date relaționale pot să apară anomalii. Procesul de optimizare a relațiilor conceptuale poartă numele de **normalizare**. Pentru a depista eventualele redundanțe se utilizează **dependențele funcționale** (**Vezi Curs 9**)
- **X → Y** - X determină funcțional pe Y sau Y depinde funcțional de X
- Anomaliile care apar în lucrul cu baza de date se produc din cauza dependențelor care există între datele din cadrul relațiilor bazei de date
 - Dependențele sunt plasate greșit în tabele!!!
- O dependență poate provoca:
 - **anomalii** la inserare, modificare sau ștergere (**Vezi Curs 9**)
 - **redundanță** în date

Necesitatea normalizării

- **Normalizarea** are drept scop:
 - suprimarea redundanței logice;
 - evitarea anomaliilor la reactualizare;
 - rezolvarea problemei reconexiunii;
- Există o **teorie matematică a normalizării** al cărei autor este E.F. Codd
 - **Soluția**: construirea unor tabele standard (forme normale);
- Normalizarea este procesul reversibil de transformare a unei relații, în mai multe relații cu structură mai simplă
 - Procesul este reversibil în sensul că nicio informație nu este pierdută în timpul transformării;
 - O relație este într-o formă normală particulară dacă ea satisface o mulțime specificată de constrângeri;

Necesitatea normalizării

➤ **Relație universală + mulțime de anomalii**

- Orice formă normală se obține aplicând o **schemă de descompunere**

Există două tipuri de descompuneri:

➤ **Descompuneri ce conservă dependențele.**

- descompunerea relației R în proiecțiile $R_1, R_2, \dots, R_k$, a.î. dependențele lui R sunt echivalente (au închideri pseudo-tranzitive identice) cu reuniunea dependențelor lui $R_1, R_2, \dots, R_k$.

➤ **Descompuneri fără pierderi de informație (*L-join*).**

- descompunerea relației R într-o mulțime de proiecții $R_1, R_2, \dots, R_j$, a.î. pentru orice instanță a lui R este adevărată relația:

$$R = \text{JOIN}(\Pi_{B_1}(R), \Pi_{B_2}(R), \dots, \Pi_{B_j}(R))$$

- Relația inițială = compunerea naturală a relațiilor obținute prin descompunere;

Necesitatea normalizării

- Formele normale sunt obținute prin descompuneri fără pierderi de informație
- O descompunere fără pierdere de informație, utilizată în procesul normalizării, este dată de **regula Casey-Delobel**:

- Fie $R(A)$ o schemă relațională și fie α, β, γ o partiție a lui A . Presupunem că α determină funcțional pe β . Atunci:

$$R(A) = \text{JOIN}(\Pi_{\alpha \cup \beta}(R), \Pi_{\alpha \cup \gamma}(R)).$$

- $\alpha \cup \beta \rightarrow$ mulțimea atributelor care intervin în dependențele funcționale;
- $\alpha \cup \gamma \rightarrow$ reprezintă reuniunea determinantului cu restul atributelor lui A .

Exemplu regula Casey-Delobel

► Se consideră relația:

AVION(A#, nume, capacitate, localitate)

$\alpha = \{\text{nume}\}$

$\beta = \{\text{capacitate}\}$

$\gamma = \{A\#, \text{localitate}\}$

Avion

A#	nume	capacitate	localitate
1	AIRBUS	250	PARIS
2	AIRBUS	250	PARIS
3	AIRBUS	250	LONDRA
4	CAR	100	PARIS
5	B707	150	LONDRA
6	B707	150	LONDRA

Regula Casey-Delobel: Fie $R(A)$ o schemă relațională și fie α , β , γ o partiție a lui A . Presupunem că α determină funcțional pe β . Atunci:

$$R(A) = \text{JOIN}(\Pi_{\alpha \cup \beta}(R), \Pi_{\alpha \cup \gamma}(R))$$

$\left\{ \begin{array}{l} \alpha \rightarrow \beta \\ \text{nume} \rightarrow \text{capacitate} \Rightarrow \text{AVION1} = (\text{nume}\#, \text{capacitate}) \end{array} \right.$

► Avem $\Pi_{\alpha \cup \beta}(R)$, unde $\alpha \cup \beta$ – este mulțimea atributelor care intervin în dependențele funcționale

► $\Pi_{\alpha \cup \gamma}(R)$, unde $\alpha \cup \gamma$ – reprezintă reuniunea determinantului cu restul atributelor relației AVION

În final, relația **AVION** se va descompune în următoarele scheme relaționale:

AVION1 = (nume#, capacitate)
AVION2 = (A#, nume, localitate)

Exemplu regula Casey-Delobel

Avion

A#	nume	capacitate	localitate
1	AIRBUS	250	PARIS
2	AIRBUS	250	PARIS
3	AIRBUS	250	LONDRA
4	CAR	100	PARIS
5	B707	150	LONDRA
6	B707	150	LONDRA



AVION1

Nume #	Capacitate
AIRBUS	250
CAR	100
B707	150

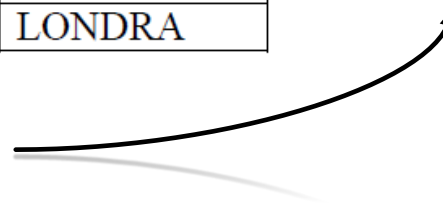
AVION2

A#	Nume	Localitate
1	AIRBUS	PARIS
2	AIRBUS	PARIS
3	AIRBUS	LONDRA
4	CAR	PARIS
5	B707	LONDRA
6	B707	LONDRA

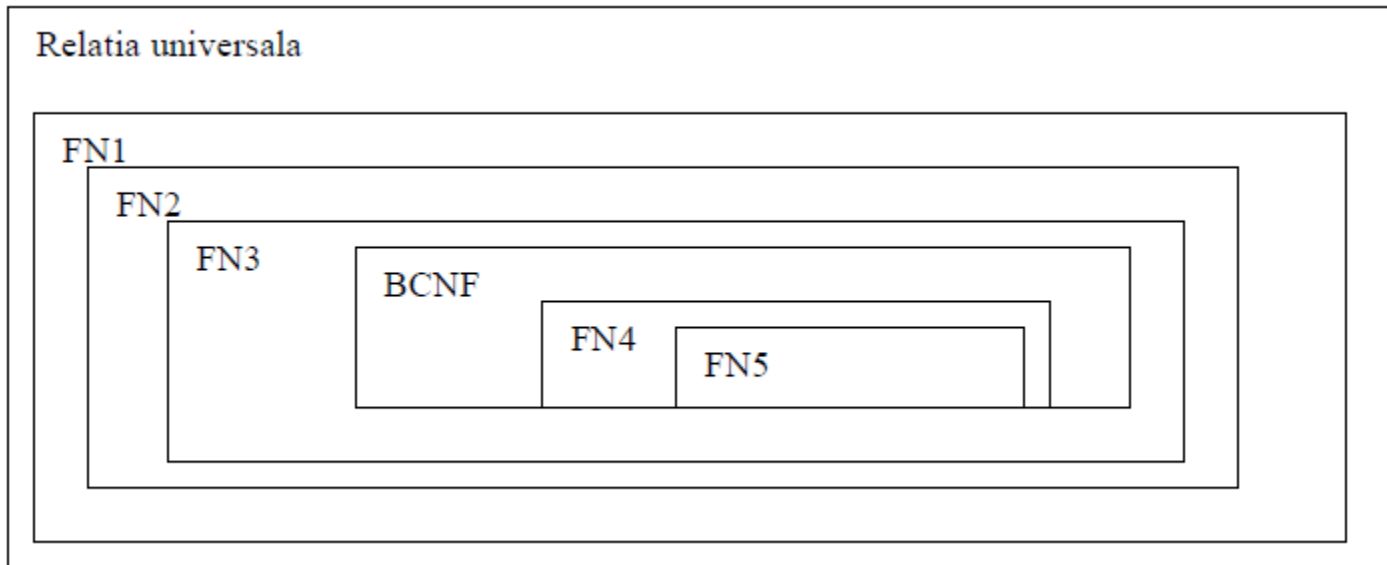
În final, relația **AVION** se va descompune în următoarele scheme relaționale:

AVION1 = (nume#, capacitate)

AVION2 = (A#, nume, localitate)



Necesitatea normalizării



Forma normală 1 (FN1)

- O relație este în **prima formă normală** dacă fiecărui atribut care o compune îi corespunde o valoare indivizibilă (atomică).
- Exemplu:

VEHICUL (Non FN1)

cod_persoana#	vehicule
P1	DL, RC, FF
P2	RM, VW
P3	DL

Forma normală 1 (FN1)

Varianta 1:

VEHICUL (FN1)

cod_persoana#	vehicul#
P1	DL
P1	RC
P1	FF
P2	RM
P2	VW
P3	DL

Varianta 2:

VEHICUL (FN1)

cod_persoana#	vehicul1	vehicul2	vehicul3
P1	DL	RC	FF
P2	RM	VW	null
P3	DL	null	null

Forma normală 2 (FN2)

- O relație R este în **a doua formă normală** dacă și numai dacă:
 - relația R este în **FN1**;
 - fiecare atribut care nu este cheie (nu participă la cheia primară) este dependent de întreaga cheie primară;
- FN2 interzice manifestarea unor dependențe funcționale parțiale în cadrul relației R

Forma normală 2 (FN2)

atasat_la

Cod_salariat#	Job_cod	Nr_proiect#	Funcția	Suma
S1	Programator	P1	Supervizor	60
S1	Programator	P2	Cercetator	25
S1	Programator	P3	Auxiliar	10
S3	Vanzator	P3	Supervizor	60
S5	Inginer	P3	Supervizor	60

- Un salariat are mai multe funcții și poate lucra la mai multe proiecte
- O relație *R* este în a doua formă normală dacă și numai dacă:
 - relația *R* este în FN1; -> **relatia atasat_la se află în FN1 deoarece avem identificator unic pentru toate intrările din tabel**
 - fiecare atribut care nu este cheie (nu participă la cheia primară) este dependent de întreaga cheie primară – în cazul nostru attributele **job_cod, funcția, suma** nu sunt chei și trebuie să depindă direct de întreaga cheie primară **cod_salariat#** și **nr_proiect#** -> aceste attribute nu depind direct de întreaga cheie primară deoarece se observă dependența directă dintre **job_cod** și **cod_salariat**, însemnând că **job_cod** depinde direct doar de o parte a cheii primare, și anume doar de **cod_salariat** -> **relația atasat_la nu se află în FN2**

Forma normală 2 (FN2)

atasat_la

Cod_salariat#	Job_cod	Nr_proiect#	Funcția	Suma
S1	Programator	P1	Supervizor	60
S1	Programator	P2	Cercetator	25
S1	Programator	P3	Auxiliar	10
S3	Vanzator	P3	Supervizor	60
S5	Inginer	P3	Supervizor	60

- Fiecare atribut care nu este cheie (nu participă la cheia primară) este dependent de întreaga cheie primară – în cazul nostru attributele **job_cod**, **funcția**, **suma** nu sunt chei și trebuie să depindă direct de întreaga cheie primară **cod_salariat#** și **nr_proiect#** -> aceste attribute nu depind direct de întreaga cheie primară deoarece se observă dependența directă dintre **job_cod** și **cod_salariat**, însemnând că **job_cod** depinde direct doar de o parte a cheii primare, și anume doar de **cod_salariat** -> **relația atasar la nu se află în FN2**
- Astfel avem:
 - {cod_salariat#} -> {job_cod} – cod_salariat determină funcțional job_cod
 - {cod_salariat#, nr_proiect#} -> {funcția, suma}

Forma normală 2 (FN2)

atasat_la

Cod_salariat#	Job_cod	Nr_proiect#	Funcția	Suma
S1	Programator	P1	Supervizor	60
S1	Programator	P2	Cercetator	25
S1	Programator	P3	Auxiliar	10
S3	Vanzator	P3	Supervizor	60
S5	Inginer	P3	Supervizor	60

➤ Astfel avem:

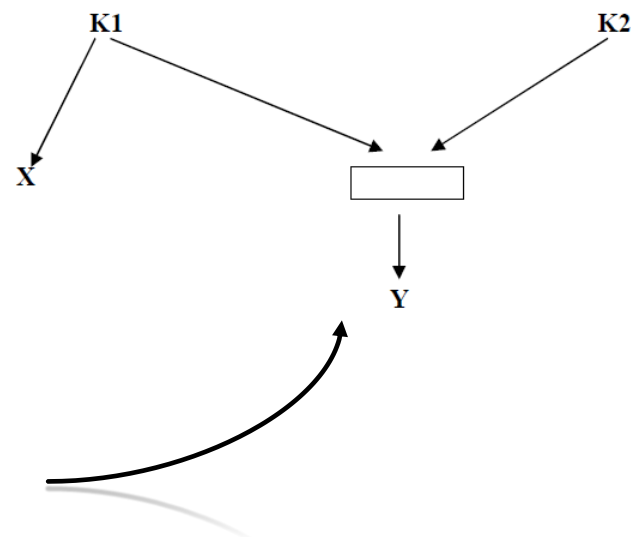
- {cod_salariat#} → {job_cod} – cod_salariat determină funcțional job_cod
- {cod_salariat#, nr_proiect#} → {funcția, suma}

Aplicarea regulii Casey-Delobel pentru FN2:

Fie relația $R(K1, K2, X, Y)$, unde $K1$ și $K2$ definesc cheia primară, iar X și Y sunt mulțimi de atribute, astfel încât $K1 \rightarrow X$.

Din cauza dependenței funcționale $K1 \rightarrow X$ care arată că R nu este în FN2, se înlocuiește R (fără pierdere de informație) cu două proiecții:

$R1(K1, K2, Y)$ și $R2(K1, X)$.



Forma normală 2 (FN2)

atasat_la

Cod_salariat#	Job_cod	Nr_proiect#	Funcția	Suma
S1	Programator	P1	Supervizor	60
S1	Programator	P2	Cercetator	25
S1	Programator	P3	Auxiliar	10
S3	Vanzator	P3	Supervizor	60
S5	Inginer	P3	Supervizor	60

Transformarea în FN2:

atasat_2a

Cod_salariat#	Nr_proiect#	Funcția	Suma
S1	P1	Supervizor	60
S1	P2	Cercetator	25
S1	P3	Auxiliar	10
S3	P3	Supervizor	60
S5	P3	Supervizor	60

atasat_2b

Cod_salariat#	Job_cod
S1	Programator
S3	Vanzator
S5	Inginer

Astfel avem:

{cod_salariat#} -> {job_cod}

– cod_salariat determină funcțional job_cod

{cod_salariat#, nr_proiect#} -> {funcția, suma}

Forma normală 2 (FN2)

Exemplu:

- Presupunem că **un șantier poate executa mai multe lucrări de bază** și că **o lucrare poate fi executată de mai multe șantiere**, iar o lucrare este definită de intervalul de timp în care este executată.

LUCRARE(cod_obiectiv#, cod_lucrare#, nume);

SANTIER(nr_santier#, specialitate, sef);

EXECUTA(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie, conducator, data_inceput, data_sfarsit).

- Pentru relația **EXECUTA** sunt evidente dependențele:

$\{\text{cod_obiectiv\#, cod_lucrare\#}\} \rightarrow \{\text{data_inceput, data_sfarsit}\},$

$\{\text{cod_obiectiv\#, cod_lucrare\#, nr_santier\#}\} \rightarrow \{\text{descriere, functie, conducator}\}.$

- Relația **EXECUTA** este în FN1, dar nu este în FN2. **Se aplică regula Casey Delobel:**
EXECUTA_1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie, conducator).
EXECUTA_2(cod_obiectiv#, cod_lucrare#, data_inceput, data_sfarsit).

Forma normală 3 (FN3)

- Intuitiv, o relație R este în **a treia formă normală** dacă și numai dacă:
 - relația R este în FN2;
 - fiecare atribut care nu este cheie (nu participă la o cheie) depinde **direct** de cheia primară;
- Fie R o relație, X o submulțime de attribute ale lui R și A un atribut al relației R . A este **dependent tranzitiv** de X dacă există Y astfel încât $X \rightarrow Y$ și $Y \rightarrow A$
 - De exemplu, dacă $K_1, K_2, K_3 \rightarrow A_1$ și dacă $K_1, K_2, A_1 \rightarrow A_2$.
 - Rezulta ca A_2 este **dependent tranzitiv** de K_1, K_2, K_3 . (A_2 este dependent de K_1, K_2, K_3 prin A_1)

Forma normală 3 (FN3)

atasat_2a

Cod_salariat#	Nr_proiect#	Functia	Suma
S1	P1	Supervizor	60
S1	P2	Cercetator	25
S1	P3	Auxiliar	10
S3	P3	Supervizor	60
S5	P3	Supervizor	60

Formal, o relație R este în **a treia formă normală** dacă și numai dacă:

- relația R este în FN2; -> **este în FN2** deoarece este în FN1 și îndeplinește condiția că fiecare atribut care nu este cheie (nu participă la cheia primară) să fie dependent de întreaga cheie primară
- fiecare atribut care nu este cheie (nu participă la o cheie) nu este dependent tranzitiv de nicio cheie a relației R. **Cu alte cuvinte o relație este în FN3 dacă și numai dacă fiecare atribut (coloană) care nu este cheie, depinde de cheie, de întreaga cheie și numai de cheie.**

În exemplul **atasat_2a** se observa că atributul **suma** depinde tranzitiv de cheia primară compusă **cod_salariat** și **nr_proiect** prin intermediul atributului **functia**.

Forma normală 3 (FN3)

atasat_2a

Cod_salariat#	Nr_proiect#	Funcția	Suma
S1	P1	Supervizor	60
S1	P2	Cercetator	25
S1	P3	Auxiliar	10
S3	P3	Supervizor	60
S5	P3	Supervizor	60

- În exemplul **atasat_2a** se observă că atributul **suma** depinde tranzitiv de cheia primară compusă **cod_salariat** și **nr_proiect** prin intermediul atributului **funcția**.
- Astfel avem:
 - {cod_salariat#, nr_proiect#} -> {funcția} – cod_salariat și nr_proiect determină funcțional atributul funcția
 - {cod_salariat#, nr_proiect#} -> {funcția} -> {suma}

Pentru a aduce relația **atasat_2a** în FN3 se aplică **regula Casey-Delobel**. Relația se descompune, prin eliminarea dependențelor funcționale tranzitive, în proiecțiile:

ATASAT_3A(cod_salariat#, nr_proiect#, funcția)

ATASAT_3B(funcția, suma)

Forma normală 3 (FN3)

atasat_2a

Cod_salariat#	Nr_proiect#	Funcția	Suma
S1	P1	Supervizor	60
S1	P2	Cercetator	25
S1	P3	Auxiliar	10
S3	P3	Supervizor	60
S5	P3	Supervizor	60

- Pentru a aduce relația **atasat_2a** în FN3, se aplică **regula Casey-Delobel**. Relația se descompune, prin eliminarea dependențelor funcționale tranzitive, în proiecțiile:

ATASAT_3A(cod_salariat#, nr_proiect#, funcția)

ATASAT_3B(funcția#, suma)

- Transformarea în FN3

atasat_3a

Cod_salariat#	Nr_proiect#	Funcția
S1	P1	Supervizor
S1	P2	Cercetator
S1	P3	Auxiliar
S3	P3	Supervizor
S5	P3	Supervizor

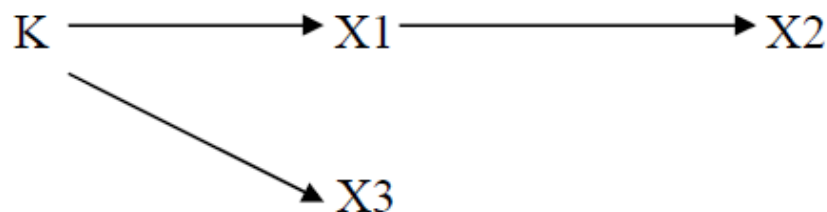
atasat_3b

Funcția#	Suma
Supervizor	60
Cercetator	25
Auxiliar	10

Forma normală 3 (FN3)

Aplicarea regulii Casey-Delobel pentru FN3:

- Fie relația $R(K, X_1, X_2, X_3)$, unde atributul X_2 depinde tranzitiv de K , iar K este cheia primară a lui R . Presupunem că $K \rightarrow X_1 \rightarrow X_2$
- Din cauza dependenței funcționale $X_1 \rightarrow X_2$ care arată că R nu este în FN3, se înlocuiește R (fără pierdere de informație) prin două proiecții $R1(K, X_1, X_3)$ și $R2(X_1, X_2)$



Forma normală 3 (FN3)

Exemplu:

- În tabelul **EXECUTA1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie, conducator)** continuă să existe redundanță în date;
- Atributul **conducator** depinde indirect de cheia primară prin intermediul atributului **functie**. Între attributele relației există dependențele:

{cod_obiectiv#, cod_lucrare#, nr_santier#} → {descriere},

{cod_obiectiv#, cod_lucrare#, nr_santier#} → {functie} → {conducator}.

- Se aplică **regula Casey-Delobel**. Relația se descompune, prin eliminarea dependențelor funcționale tranzitive, în proiecțiile:

EXECUTA1_1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie)

EXECUTA1_2(functie, conducator).

Schema de sinteză pentru obținerea lui FN3

- **Algoritmul de sinteză** construiește o acoperire minimală F a dependențelor funcționale totale.
 - Se elimină atributele și dependențele funcționale redundante;
 - Mulțimea F este partiționată în grupuri F_i , astfel încât în fiecare grup F_i sunt dependențe funcționale care au același membru stâng și nu există două grupuri având același membru stâng;
 - Fiecare grup F_i produce o schemă FN3;
- Algoritmul realizează o descompunere ce conservă dependențele.

Schema de sinteză pentru obținerea lui FN3

Algoritm SNF3 (aducerea unei relații în FN3 prin utilizarea unei scheme de sinteză):

1. Se determină **F o acoperire minimală a lui E** (mulțimea dependențelor funcționale);
2. Se descompune mulțimea F în **grupuri notate F_i** , astfel încât în cadrul fiecărui grup să existe dependențe funcționale **având aceeași parte stângă**;
3. Se determină **perechile de chei echivalente (X, Y)** în raport cu F (două mulțimi de attribute X, Y sunt chei echivalente dacă în mulțimea de dependențe E există atât dependența $X \rightarrow Y$, cât și dependența $Y \rightarrow X$);

Schema de sinteză pentru obținerea lui FN3

4. Pentru fiecare pereche de chei echivalente: se identifică grupurile F_i și F_j care conțin dependențele funcționale cu partea stângă X și respectiv Y ; se formează **un nou grup de dependențe F_{ij}** , care va conține dependențele funcționale având membrul stâng (X, Y) ; se elimină grupurile F_i și F_j , iar locul lor va fi luat de grupul F_{ij} .
5. Se determină o **acoperire minimală a lui F** , care va include toate dependențele $X \rightarrow Y$, unde X și Y sunt chei echivalente (celelalte dependențe sunt redundante).
6. Se construiesc relații FN3 (câte o relație pentru fiecare grup de dependențe funcționale).

Schema de sinteză pentru obținerea lui FN3

- Se observă că algoritmul solicită:
 - determinarea unei **acoperiri minimale** (algoritmii **EAR** și **EDF**);
 - determinarea **închiderii (A^+) unei mulțimi de attribute A** în raport cu mulțimea de dependențe funcționale E (algoritm **AIDF**).
- Determinarea acoperirii minimale presupune eliminarea atributelor și dependențelor redundante;
- Acoperirea minimală **nu este unică** și depinde de ordinea în care sunt eliminate aceste attribute și dependențe redundante;

Schema de sinteză pentru obținerea lui FN3

- **Algoritm EAR** (elimină attributele redundante din determinantul dependențelor funcționale)
- Pentru fiecare dependență funcțională din E și pentru fiecare atribut din partea stângă a unei dependențe funcționale:
 - **Pas1.** Se elimină atributul considerat.
 - **Pas2.** Se calculează închiderea părții stângi reduse.
 - **Pas3.** Dacă închiderea conține toate attributele din determinantul dependenței, atunci atributul eliminat la pasul 1 este redundant și rămâne eliminat. În caz contrar, atributul nu este redundant și se reintroduce în partea stângă a dependenței funcționale.

Schema de sinteză pentru obținerea lui FN3

- **Algoritm EDF** (elimină dependențele funcționale redundante din E)
- Pentru fiecare dependență funcțională $X \rightarrow Y$ din E:
 - **Pas1.** Se elimină dependența din E.
 - **Pas2.** Se calculează închiderea X^+ , în raport cu mulțimea redusă de dependențe.
 - **Pas3.** Dacă Y este inclus în X^+ , atunci dependența $X \rightarrow Y$ este redundantă și rămâne eliminată. În caz contrar, se reintroduce în E.

Schema de sinteză pentru obținerea lui FN3

➤ **Algoritm AIDF** (determină închiderea lui A)

- **Pas1.** Se caută dacă există în E dependențe $X \rightarrow Y$ pentru care determinantul X este o submulțime a lui A, iar determinatul Y nu este inclus în A.
- **Pas2.** Pentru fiecare astfel de dependență funcțională se adaugă mulțimii A attributele care constituie determinatul dependenței.
- **Pas3.** Dacă nu mai există dependențe funcționale de tipul celor de la pasul 1, atunci $A^+ = A$.

Exemplul 1

Fie relația **R(F, N, P, U, C, T)**. Se aplică **algoritmul SNF3** (aducerea unei relații în FN3 prin utilizarea schemei de sinteză).

1. Se determină **F o acoperire minimală a lui E** (mulțimea dependențelor funcționale). Aceste dependențe se aplică știind regulile de proiectare ale modelului. Se pornește de la descrierea modelului real.

$F \rightarrow N$; $F \rightarrow P$; $P, F, N \rightarrow U$; $P \rightarrow C$; $P \rightarrow T$; $C \rightarrow T$; $N \rightarrow F$

Exemplul 1

2. Se descompune mulțimea F în **grupuri notate F_i** , astfel încât în cadrul fiecărui grup să existe dependențe funcționale **având aceeași parte stângă**;

F1: $F \rightarrow N$

F2: $F \rightarrow P$

F3: $P, F, N \rightarrow U$

F4: $P \rightarrow C$

F5: $P \rightarrow T$

F6: $C \rightarrow T$

F7: $N \rightarrow F$

Exemplul 1

Se elimină **atributele** și dependențele funcționale redundante.

3. Se determină **perechile de chei echivalente (X, Y)** în raport cu F (două mulțimi de atribute X, Y sunt chei echivalente dacă în mulțimea de dependențe E există atât dependența $X \rightarrow Y$, cât și dependența $Y \rightarrow X$);

F1: $F \rightarrow N$

F2: $F \rightarrow P$

F3: $P, F, N \rightarrow U$

F4: $P \rightarrow C$

F5: $P \rightarrow T$

F6: $C \rightarrow T$

F7: $N \rightarrow F$

➤ **Atributele redundante** se identifică în raport cu dependențele pe care le au cu alte atribute.

În **F3** se observă dependența **$P \rightarrow U$**
- se poate elimina astfel atributul P deoarece avem:

➤ **F2:** $F \rightarrow P$ și

➤ **F3:** $P \rightarrow U$

➤ rezultând **$F \rightarrow U$** , dependență pe care o avem deja tot în **F3**.

Exemplul 1

Se elimină **atributele** și dependențele funcționale redundante.

F1: $F \rightarrow N$

F2: $F \rightarrow P$

F3: $P, F, N \rightarrow U$

F4: $P \rightarrow C$

F5: $P \rightarrow T$

F6: $C \rightarrow T$

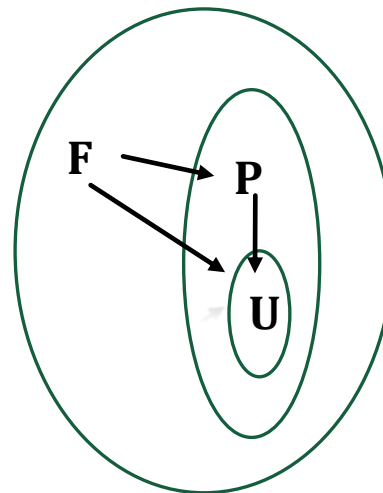
F7: $N \rightarrow F$

Atributul se elimină utilizând pseudo-transitivitatea astfel:

dacă $X \rightarrow Y$ și $W \cup Y \rightarrow Z$, atunci $W \cup X \rightarrow Z$

$F \rightarrow P$ și $P, F \rightarrow U$, atunci rezultă $F \rightarrow U$

SAU conform reprezentării grafice de mai jos:



Exemplul 1

Se elimină **atributele** și dependențele funcționale redundante.

F1: $F \rightarrow N$

F2: $F \rightarrow P$

F3: $P, F, N \rightarrow U$

F4: $P \rightarrow C$

F5: $P \rightarrow T$

F6: $C \rightarrow T$

F7: $N \rightarrow F$

La fel se procedează și în cazul atributului N

$F \rightarrow N$

$F, N \rightarrow U$

=> se elimină N și se păstrează în F3 doar dependența $F \rightarrow U$

Exemplul 1

Se elimină **atributele** și dependențele funcționale redundante.

F1: $F \rightarrow N$

F2: $F \rightarrow P$

F3: $P, F, N \rightarrow U$

F4: $P \rightarrow C$

F5: $P \rightarrow T$

F6: $C \rightarrow T$

F7: $N \rightarrow F$

În final, F3 o să aibă dependența funcțională
 $F \rightarrow U$

Exemplul 1

Se elimină attributele și **dependențele funcționale** redundante.

3. Se determină **perechile de chei echivalente (X, Y)** în raport cu F (două mulțimi de attribute X, Y sunt chei echivalente dacă în mulțimea de dependențe E există atât dependența $X \rightarrow Y$, cât și dependența $Y \rightarrow X$);

F1: $F \rightarrow N$

F2: $F \rightarrow P$

F3: $F \rightarrow U$

F4: $P \rightarrow C$

F5: $P \rightarrow T$

F6: $C \rightarrow T$

F7: $N \rightarrow F$

În acest caz, se identifică perechi echivalente, analizând dependențele funcționale marcate cu roșu.

Din dependențele $P \rightarrow C$ și $C \rightarrow T$ rezultă că $P \rightarrow T$.

Deci **F5: $P \rightarrow T$** este o dependență funcțională care poate fi eliminată.

Exemplul 1

Se elimină attributele și **dependențele funcționale** redundante.

3. Se determină **perechile de chei echivalente** (X, Y) în raport cu F (două mulțimi de attribute X, Y sunt chei echivalente dacă în mulțimea de dependențe E există atât dependența $X \rightarrow Y$, cât și dependența $Y \rightarrow X$);

F1: $F \rightarrow N$

F2: $F \rightarrow P$

F3: $F \rightarrow U$

F4: $P \rightarrow C$

~~**F5:** $P \rightarrow T$~~

F6: $C \rightarrow T$

F7: $N \rightarrow F$

Pentru eliminarea dependențelor funcționale se identifică perechi echivalente. În acest caz, se analizează dependențele funcționale marcate cu roșu.

Din dependențele **$P \rightarrow C$** și **$C \rightarrow T$** rezultă că **$P \rightarrow T$** .

Deci **$F5: P \rightarrow T$** este o dependență funcțională care poate fi eliminată.

Exemplul 1

După eliminarea atributelor și dependențelor funcționale, se aplică următoarea regulă de sinteză:

Mulțimea F este partiționată în grupuri F_i , astfel încât în fiecare grup F_i sunt dependențe funcționale care au același membru stâng și nu există două grupuri având același membru stâng.

Am renumerotat dependențele rămase în urma eliminărilor efectuate la pași anteriori, după care vom aplica regula de mai sus.

G1: **F** $\rightarrow$ N

G2: **F** $\rightarrow$ P

G3: **F** $\rightarrow$ U

G4: **P** $\rightarrow$ C

G5: **C** $\rightarrow$ T

G6: **N** $\rightarrow$ F

Se observă dependențele care au același membru stâng. Pe acestea le vom regrupa.

Exemplul 1

Mulțimea F este partiționată în grupuri F_i , astfel încât în fiecare grup F_i sunt dependențe funcționale care au același membru stâng și nu există două grupuri având același membru stâng.

G1: **F** $\rightarrow$ N

G2: **F** $\rightarrow$ P

G3: **F** $\rightarrow$ U

G4: **P** $\rightarrow$ C

G5: **C** $\rightarrow$ T

G6: **N** $\rightarrow$ F

F1 = {G1, G2, G3}

F2 = {G4}

F3 = {G5}

F4 = {G6}

Exemplul 1

4. Pentru fiecare pereche de chei echivalente: se identifică grupurile F_i și F_j care conțin dependențele funcționale cu partea stângă X și respectiv Y ; se formează **un nou grup de dependențe F_{ij}** , care va conține dependențele funcționale având membrul stâng (X, Y) ; se elimină grupurile F_i și F_j , iar locul lor va fi luat de grupul F_{ij} .

G1: **F** -> N

G2: **F** -> P

G3: **F** -> U

G4: **P** -> C

G5: **C** -> T

G6: **N** -> F

F1 = {G1, G2, G3}

F2 = {G4}

F3 = {G5}

F4 = {G6}



F1 = {G1, G2, G3, **G6**}

F2 = {G4}

F3 = {G5}

~~F4 = {G6}~~

G1 și G6 sunt
echivalente

Exemplul 1

5. Se determină o **acoperire minimală a lui F** , care va include toate dependențele $X \rightarrow Y$, unde X și Y sunt chei echivalente (celelalte dependențe sunt redundante).
6. Se construiesc relații FN3 (câte o relație pentru fiecare grup de dependențe funcționale).

$F1 = \{G1, G2, G3, G6\}$

$F2 = \{G4\}$

$F3 = \{G5\}$



$R1 = \{F\#, N, P, U\}$

$R2 = \{P\#, C\}$

$R3 = \{C\#, T\}$

G1: $F \rightarrow N$

G2: $F \rightarrow P$

G3: $F \rightarrow U$

G4: $P \rightarrow C$

G5: $C \rightarrow T$

G6: $N \rightarrow F$

Exemplul 2

Se dă următoarea schemă relațională. Stiind că **descrierea** se realizează în funcție de **obiectiv**, de **lucrarea executată** și de **șantierul** pe care aceasta are loc, iar **conducătorul** depinde de **funcția** pe care o are, aplicați **algoritmul de sinteză FN3** (SNF3) și aduceți relația în forma normală 3.

EXECUTA1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie, conducator)

Se notează attributele relației **EXECUTA1**:

cod_obiectiv = A

cod_lucrare = B

nr_santier = C

descriere = D

functie = E

conducator = F

O relație este în **FN3** dacă și numai dacă fiecare atribut (coloană) care nu este cheie, depinde de cheie, de întreaga cheie și numai de cheie.

Exemplul 2

Se dă următoarea schemă relațională. Stiind că **descrierea** se realizează în funcție de **obiectiv**, de **lucrarea executată** și de **șantierul** pe care aceasta are loc, iar **conducătorul** depinde de **funcția** pe care o are, aplicați **algoritmul de sinteză FN3** (SNF3) și aduceți relația în forma normală 3.

EXECUTA1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie, conducator)

1. Se determină **F o acoperire minimală a lui E** (mulțimea dependențelor funcționale). Aceste dependențe se aplică știind regulile de proiectare ale modelului. Se pornește de la descrierea modelului real.
2. Se descompune mulțimea F în **grupuri notate F_i** , astfel încât în cadrul fiecărui grup să existe dependențe funcționale **având aceeași parte stângă**;

Exemplul 2

Se dă următoarea schemă relațională. Stiind că **descrierea** se realizează în funcție de **obiectiv**, de **lucrarea executată** și de **șantierul** pe care aceasta are loc, iar **conducătorul** depinde de **funcția** pe care o are, aplicați **algoritmul de sinteză FN3** (SNF3) și aduceți relația în forma normală 3.

EXECUTA1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie, conducator)

F1: A, B, C -> D

F2: A, B, C -> E

F3: E -> F

F1: cod_obiectiv, cod_lucrare, nr_santier -> descriere

F2: cod_obiectiv, cod_lucrare, nr_santier -> functie

F3: functie -> conducator

Exemplul 2

Se elimină attributele și dependențele funcționale redundante.

În exemplul curent nu există attribute sau dependențe funcționale redundante. Așadar, se trece la pașii următori.

EXECUTA1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie, conducator)

F1: A, B, C -> D

F1: cod_obiectiv, cod_lucrare, nr_santier -> descriere

F2: A, B, C -> E

F2: cod_obiectiv, cod_lucrare, nr_santier -> functie

F3: E -> F

F3: functie -> conducator

Exemplul 2

Mulțimea F este partiționată în grupuri F_i , astfel încât în fiecare grup F_i sunt dependențe funcționale care au același membru stâng și nu există două grupuri având același membru stâng.

4. Pentru fiecare pereche de chei echivalente: se identifică grupurile F_i și F_j care conțin dependențele funcționale cu partea stângă X și respectiv Y ; se formează **un nou grup de dependențe F_{ij}** , care va conține dependențele funcționale având membrul stâng (X, Y) ; se elimină grupurile F_i și F_j , iar locul lor va fi luat de grupul F_{ij} .

EXECUTA1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie, conducator)

F1: A, B, C -> D

F2: A, B, C -> E

F3: E -> F



$F_{1,2} = \{A, B, C, D, E\}$

$F3 = \{E, F\}$



$F_{1,2} = \{\text{cod_obiectiv}, \text{cod_lucrare}, \text{nr_santier}, \text{descriere}, \text{functie}\}$

$F3 = \{\text{functie}, \text{conducator}\}$

Exemplul 2

5. Se determină o **acoperire minimală a lui F** , care va include toate dependențele $X \rightarrow Y$, unde X și Y sunt chei echivalente (celelalte dependențe sunt redundante).
6. Se construiesc relații FN3 (câte o relație pentru fiecare grup de dependențe funcționale).

EXECUTA1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie, conducator)

$F_{1,2} = \{\text{cod_obiectiv}, \text{cod_lucrare}, \text{nr_santier}, \text{descriere}, \text{functie}\}$

$F_3 = \{\text{functie}, \text{conducator}\}$



EXECUTA1_1(cod_obiectiv#, cod_lucrare#, nr_santier#, descriere, functie)

EXECUTA1_2(functie#, conducator).